

Data Structures

Lists, Stacks and Queues

DIGITAL AND PUBLIC POLICY STREAM
Python Tutorial

Today's focus

You already use lists, loops and functions.

Today: choose list methods, manage a stack and process a queue.

Predict the result before running each example.

Python 3

```
# A familiar starting point
scores = [42, 76, 50]
for score in scores:
    if score >= 50:
        print(score)
```

Output

```
76
50
```

append() and extend()

What happens when the new item is itself a list?

Compare the two results.

Use extend when each incoming item should become a separate element.

Python 3

```
a = [1, 2]
a.append([3, 4])
print(a)
```

```
b = [1, 2]
b.extend([3, 4])
print(b)
```

Output

```
[1, 2, [3, 4]]
[1, 2, 3, 4]
```

insert() and remove()

insert uses a position.

remove searches for a value.

With duplicates, which matching value disappears?

A missing value causes ValueError.

Python 3

```
names = ['Ana', 'Bo', 'Ana']
names.insert(1, 'Cam')
print(names)
names.remove('Ana')
print(names)
```

Output

```
['Ana', 'Cam', 'Bo', 'Ana']
['Cam', 'Bo', 'Ana']
```

pop() and clear()

pop removes and returns an item.
Its default position is the end.

clear empties the existing list.

pop on an empty list raises IndexError.

Python 3

```
items = [10, 20, 30]
last = items.pop()
print(last, items)
first = items.pop(0)
print(first, items)
items.clear()
print(items)
```

Output

```
30 [10, 20]
10 [20]
[]
```

Finding and counting values

Use count for frequency and index for the first matching position.

What happens when the value is absent?

Guard an index lookup with in when absence is expected.

Python 3

```
votes = ['yes', 'no', 'yes']
print(votes.count('yes'))
print(votes.index('no'))
if 'maybe' in votes:
    print(votes.index('maybe'))
```

Output

2

1

Sorting and reversing

Trace the same list through both operations.

reverse changes the current order.

For descending sorted order, use `sort(reverse=True)`.

Python 3

```
values = [3, 1, 2]
values.reverse()
print(values)
values.sort()
print(values)
values.sort(reverse=True)
print(values)
```

Output

```
[2, 1, 3]
[1, 2, 3]
[3, 2, 1]
```

Sorting with a key function

Use a function to choose what Python compares.

Here, len supplies each word's length.

We pass len itself, without calling it.

The original list stays unchanged with sorted().

Python 3

```
words = ['Rome', 'Oslo', 'Lyon']  
ordered = sorted(words, key=len)  
print(ordered)
```

```
words = ['Paris', 'NY', 'Rome']  
words.sort(key=len)  
print(words)
```

Output

```
['Rome', 'Oslo', 'Lyon']  
['NY', 'Rome', 'Paris']
```

Assignment and shallow copies

Assignment shares the same list.
copy creates a new outer list.

Nested mutable items remain shared.

Predict both changes before running the code.

Python 3

```
a = [[1], [2]]  
b = a.copy()  
b.append([3])  
b[0].append(9)  
print(a)  
print(b)
```

Output

```
[[1, 9], [2]]  
[[1, 9], [2], [3]]
```

Mutation and return values

A common mistake:
`values = values.sort()`

This replaces `values` with `None`.

Keep the mutation on its own line.
Use `sorted(values)` for a new list.

Python 3

```
values = [3, 1, 2]
result = values.sort()
print(values)
print(result)
```

```
removed = values.pop()
print(removed)
```

Output

```
[1, 2, 3]
None
3
```

Quick check: list methods

Predict a, b and x.

Which line changes both a and b?

Then replace `b = a` with `b = a.copy()`. What changes?

Python 3

```
a = [3, 1, 3]
b = a
b.remove(3)
x = a.pop()
b.extend([2, 4])
b.sort()
print(a, b, x)
```

Quick check: solution

With `b = a`, both names refer to the same list.

With a copy, removing from `b` leaves `a` unchanged.

In both versions, `x` is 3.

Python 3

```
# With b = a
# a and b both become:
[1, 2, 4]
```

```
# With b = a.copy()
# a becomes [3, 1]
# b becomes [1, 2, 3, 4]
```

Lists as stacks

A stack is last in, first out: LIFO.

Think of undoing the most recent action.

Treat the right end as the top.
append adds; pop retrieves.

Python 3

```
history = []
history.append('type A')
history.append('type B')
print(history.pop())
print(history)
```

Output

```
type B
['type A']
```

A safe stack operation

This function mutates the supplied list.

The empty-list check prevents `IndexError`.

`None` represents an empty stack here.
Use this convention when `None` is not a valid stored action.

Python 3

```
def undo(history):  
    if history:  
        return history.pop()  
    return None  
  
actions = ['A', 'B']  
print(undo(actions))  
print(actions)
```

Output

```
B  
['A']
```

Lists as queues

A queue is first in, first out: FIFO.

Think of processing requests in arrival order.

`pop(0)` works, but shifts the remaining items.

For repeated front removals, use `deque`.

Python 3

```
waiting = ['Ana', 'Bo']  
waiting.append('Cam')  
served = waiting.pop(0)  
print(served)  
print(waiting)
```

Output

```
Ana  
['Bo', 'Cam']
```

A queue with deque

deque comes from Python's standard library. No installation is needed.

append adds at the right.
popleft retrieves from the left.

An empty popleft raises IndexError.

Python 3

```
from collections import deque

waiting = deque(['Ana', 'Bo'])
waiting.append('Cam')
print(waiting.popleft())
print(list(waiting))
```

Output

```
Ana
['Bo', 'Cam']
```

Stack or queue?

Choose according to the required retrieval order.

Undo latest action: stack.

Serve earliest request: queue.

Both examples begin with A, B, C.

Python 3

```
from collections import deque

stack = ['A', 'B', 'C']
queue = deque(['A', 'B', 'C'])
print(stack.pop())
print(queue.popleft())
```

Output

C
A

Hands-on: a request queue

Write `serve_next(waiting)`.

Return and remove the oldest request.

Return `None` if the queue is empty.

Add C, call the function twice, then inspect the remaining queue.

Also test an empty deque.

Python 3

```
from collections import deque
```

```
waiting = deque(['A', 'B'])
```

```
def serve_next(waiting):  
    # Complete this function  
    pass
```

Hands-on: solution

The argument refers to the caller's deque.

popleft changes that deque.

Serving twice retrieves A, then B.
C remains for the next call.

Python 3

```
def serve_next(waiting):  
    if waiting:  
        return waiting.popleft()  
    return None
```

```
waiting = deque(['A', 'B'])  
waiting.append('C')  
print(serve_next(waiting))  
print(serve_next(waiting))  
print(list(waiting))
```

Output

```
A  
B  
['C']
```

Recap and readings

Choose a method by its effect and its return value.

A copy may still share nested items.

Use a list for a stack and deque for a queue.

Reading: Python tutorial, sections 5.1, 5.1.1 and 5.1.2.

Python 3

```
# Stack
stack.append(item)
item = stack.pop()

# Queue
queue.append(item)
item = queue.popleft()
```